



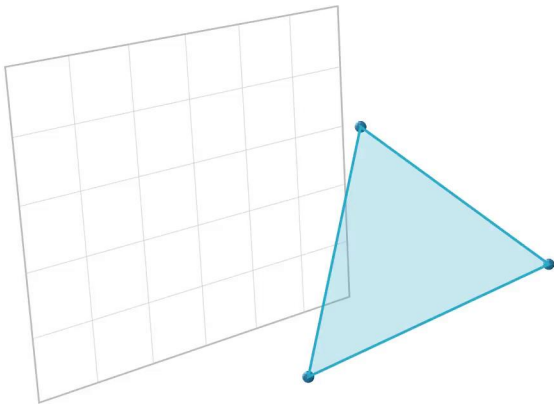
SoftJAX & SoftTorch

Empowering Automatic Differentiation Libraries with Informative Gradients

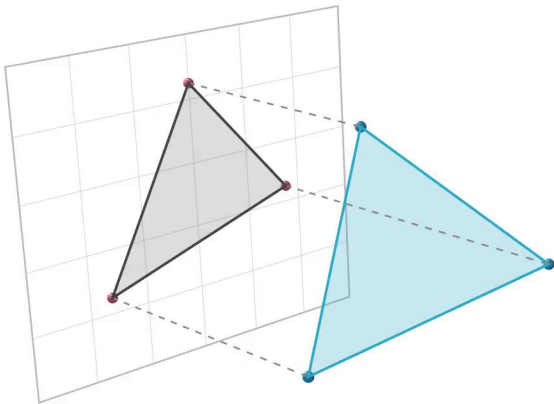
A. René Geist - June 2026, AI4PEX

Authors: Anselm Paulus* A. René Geist* Vít Musil Sebastian Hoffmann Georg Martius

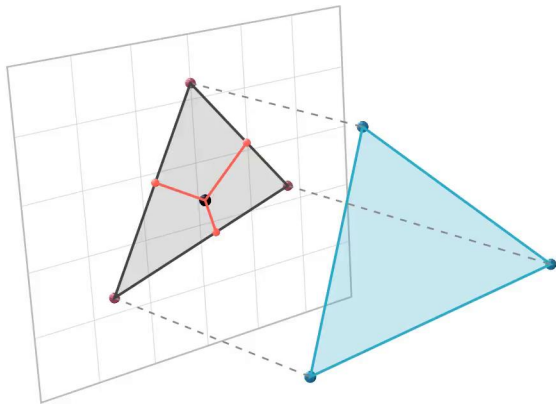
Example: Differentiable rendering



Example: Differentiable rendering



Example: Differentiable rendering

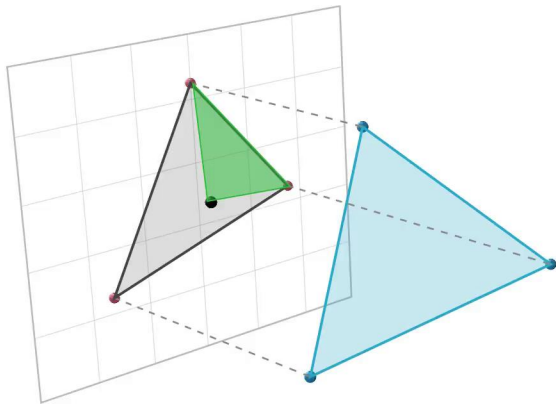


Check if triangle covers pixel

Distance-based:

```
d_min = jnp.min(jnp.array([d1, d2, d3]))  
inside = ( min_dist > 0.0 )
```

Example: Differentiable rendering



Check if triangle covers pixel

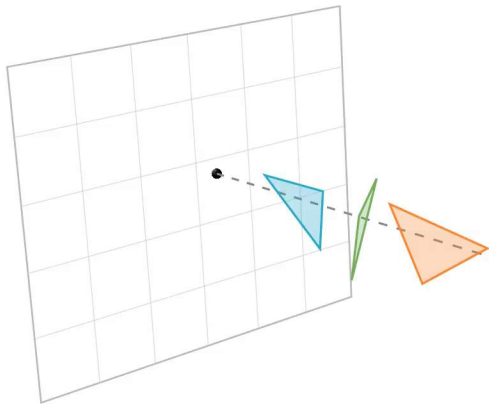
Distance-based:

```
d_min = jnp.min(jnp.array([d1, d2, d3]))  
inside = ( min_dist > 0.0 )
```

Area-based:

```
inside = jnp.all( [a1 > 0.0, a2 > 0.0, a3 > 0] )
```

Example: Differentiable rendering



Check if triangle covers pixel

Distance-based:

```
d_min = jnp.min(jnp.array([d1, d2, d3]))  
inside = ( min_dist > 0.0 )
```

Area-based:

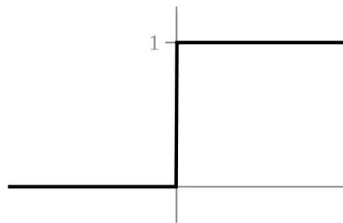
```
inside = jnp.all( [a1 > 0.0, a2 > 0.0, a3 > 0] )
```

Determine closest triangle

e.g. using argmax, sort, top_k

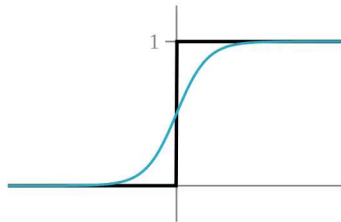
Softening

$$H(x) = \begin{cases} 0 & x < 0 \\ 0.5 & x = 0 \\ 1 & x > 0 \end{cases}$$



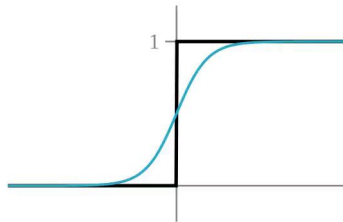
Softening

$$H_{\tau}(x) = \frac{1}{1 + e^{-x/\tau}}$$



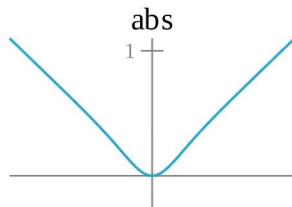
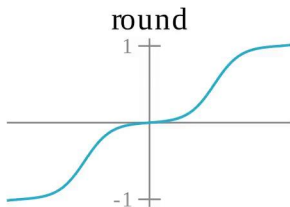
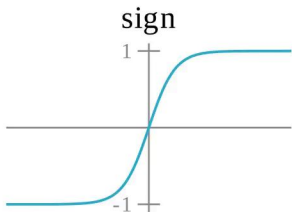
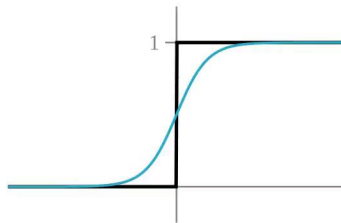
Softening

$$H_{\tau}(x) = \frac{1}{1 + e^{-x/\tau}}$$



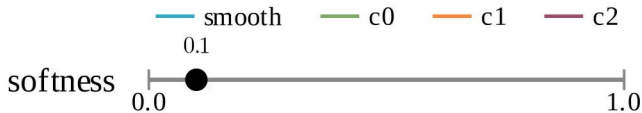
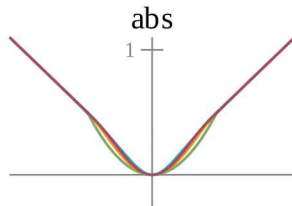
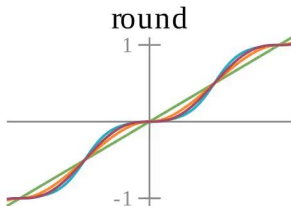
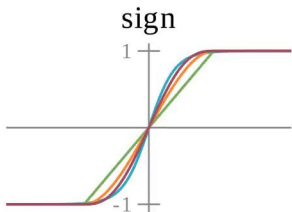
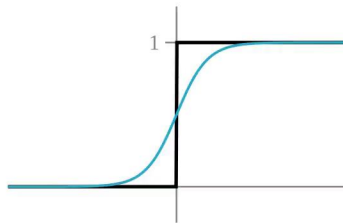
Softening

$$H_{\tau}(x) = \frac{1}{1 + e^{-x/\tau}}$$



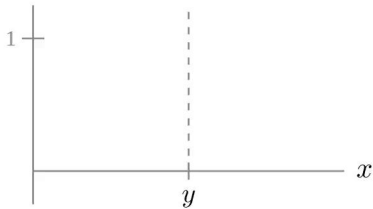
Softening

$$H_{\tau}(x) = \frac{1}{1 + e^{-x/\tau}}$$

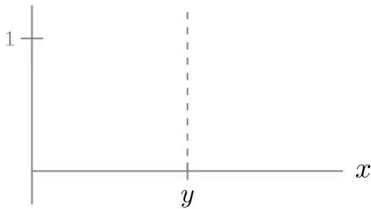


Fuzzy logic

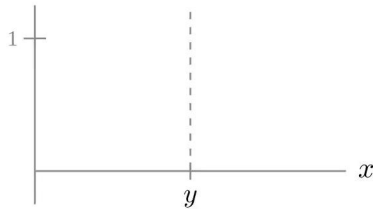
`sj.greater(x, y)`



`sj.less_equal(x, y)`

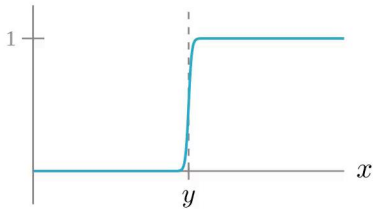


`sj.isclose(x, y)`

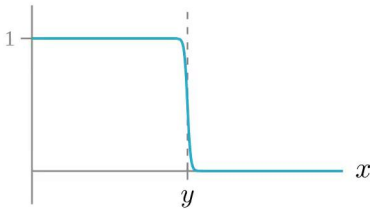


Fuzzy logic

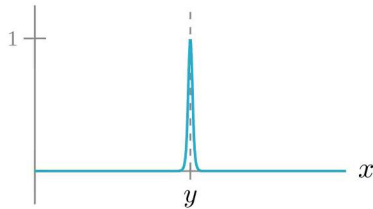
`sj.greater(x, y)`



`sj.less_equal(x, y)`

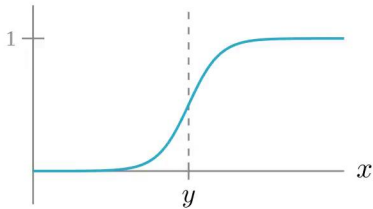


`sj.isclose(x, y)`

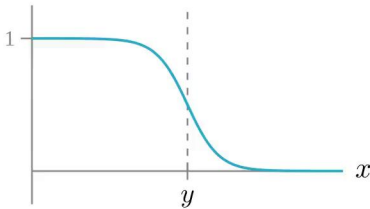


Fuzzy logic

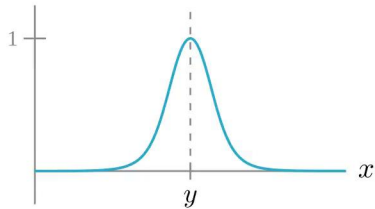
`sj.greater(x, y)`



`sj.less_equal(x, y)`

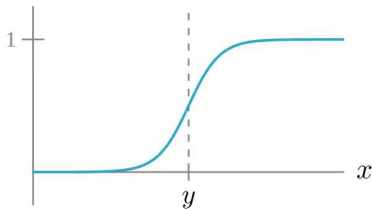


`sj.isclose(x, y)`

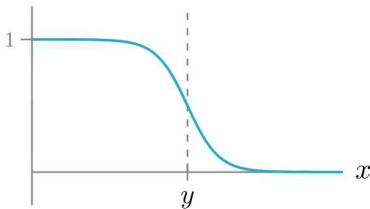


Fuzzy logic

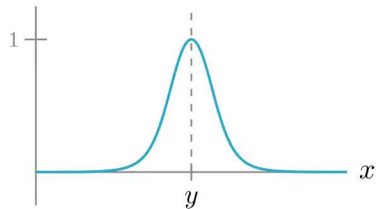
`sj.greater(x, y)`



`sj.less_equal(x, y)`



`sj.isclose(x, y)`

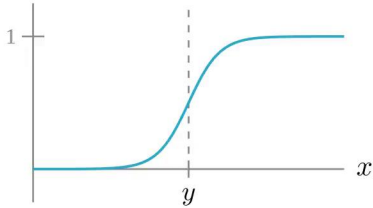


CDF defining the probability that a Boolean operator equates to true.

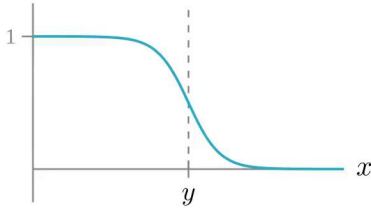


Fuzzy logic

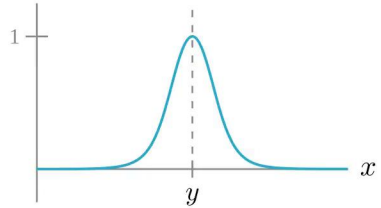
`sj.greater(x, y)`



`sj.less_equal(x, y)`



`sj.isclose(x, y)`

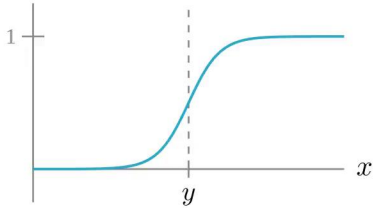


$$\text{all}(p_1, \dots, p_n) = \prod_i p_i$$

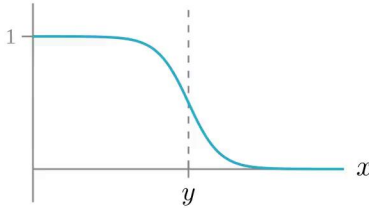
$$\text{not}(p) = 1 - p$$

Fuzzy logic

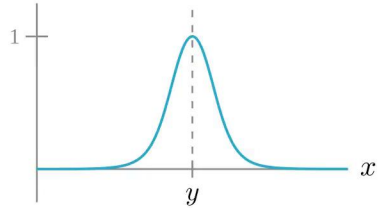
`sj.greater(x, y)`



`sj.less_equal(x, y)`



`sj.isclose(x, y)`



$$\text{all}(p_1, \dots, p_n) = \prod_i p_i$$

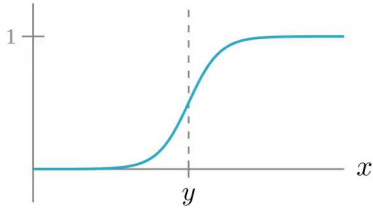
$$\text{not}(p) = 1 - p$$



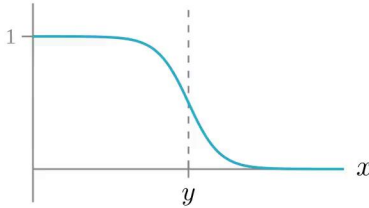
any
and
or
xor

Fuzzy logic

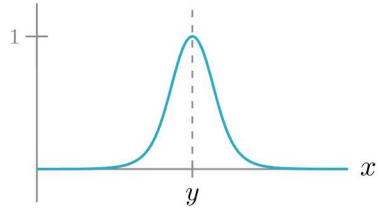
`sj.greater(x, y)`



`sj.less_equal(x, y)`



`sj.isclose(x, y)`



$$\text{all}(p_1, \dots, p_n) = \prod_i p_i$$

$$\text{not}(p) = 1 - p$$



any
and
or
xor

Selection = Expectation

$$z_i = p_i \cdot x_i + (1 - p_i) \cdot y_i$$

Axiswise operators

Standard indexing

```
x = jnp.array([0.1, 0.4, 0.8])  
idx = jnp.argmax(x)  
y = x[idx]
```

Axiswise operators

Standard indexing

```
x = jnp.array([0.1, 0.4, 0.8])  
idx = jnp.argmax(x)  
y = x[idx]
```



Index: 2, Value: 0.8

Axiswise operators

Standard indexing

```
x = jnp.array([0.1, 0.4, 0.8])  
idx = jnp.argmax(x)  
y = x[idx]
```



Index: 2, Value: 0.8

SoftJax, mode="hard"

```
hard_idx = sj.argmax(x, mode="hard")  
y = sj.dynamic_index_in_dim(x, hard_idx)
```

Axiswise operators

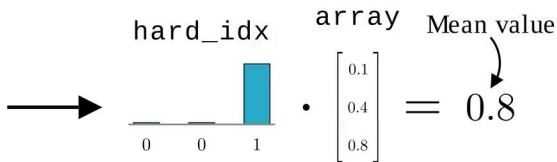
Standard indexing

```
x = jnp.array([0.1, 0.4, 0.8])  
idx = jnp.argmax(x)  
y = x[idx]
```

→ Index: 2, Value: 0.8

SoftJax, mode="hard"

```
hard_idx = sj.argmax(x, mode="hard")  
y = sj.dynamic_index_in_dim(x, hard_idx)
```



Axiswise operators

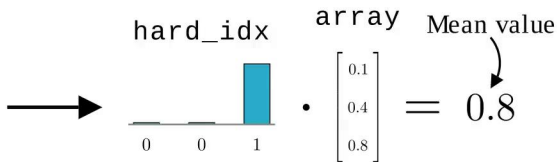
Standard indexing

```
x = jnp.array([0.1, 0.4, 0.8])  
idx = jnp.argmax(x)  
y = x[idx]
```

→ Index: 2, Value: 0.8

SoftJax, mode="hard"

```
hard_idx = sj.argmax(x, mode="hard")  
y = sj.dynamic_index_in_dim(x, hard_idx)
```



SoftJax, mode="soft"

```
soft_idx = sj.argmax(x)  
y = sj.dynamic_index_in_dim(x, soft_idx)
```

Axiswise operators

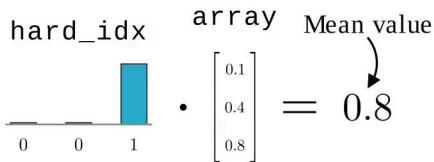
Standard indexing

```
x = jnp.array([0.1, 0.4, 0.8])  
idx = jnp.argmax(x)  
y = x[idx]
```

→ Index: 2, Value: 0.8

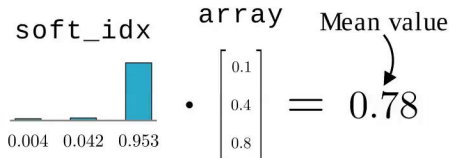
SoftJax, mode="hard"

```
hard_idx = sj.argmax(x, mode="hard")  
y = sj.dynamic_index_in_dim(x, hard_idx)
```

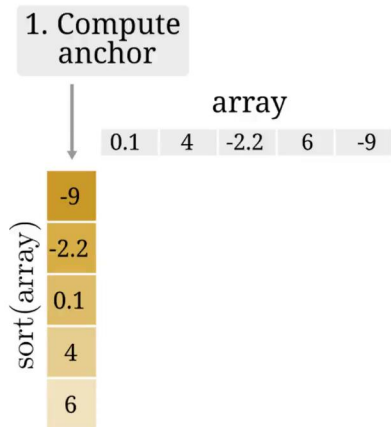


SoftJax, mode="soft"

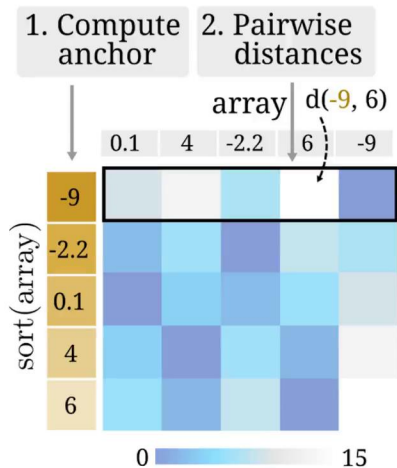
```
soft_idx = sj.argmax(x)  
y = sj.dynamic_index_in_dim(x, soft_idx)
```



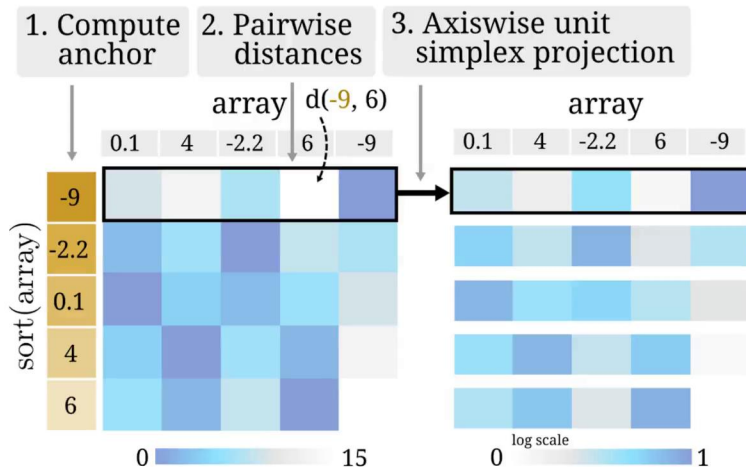
Example: SoftSort



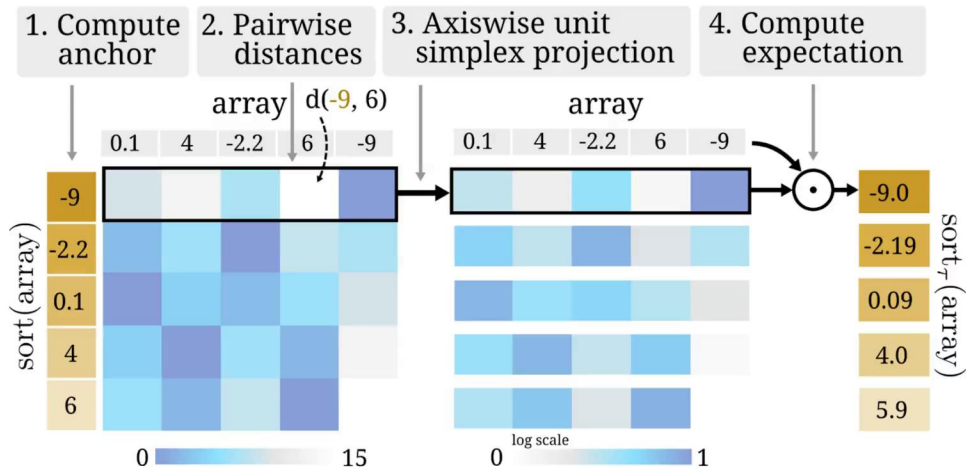
Example: SoftSort



Example: SoftSort



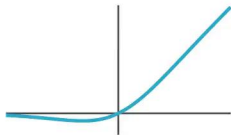
Example: SoftSort



Connection between ReLu and axiswise operators

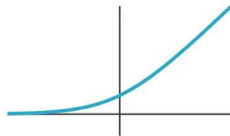
Gating

$$\text{relu}_\tau(x) := H_\tau(x) \cdot x$$



Integration

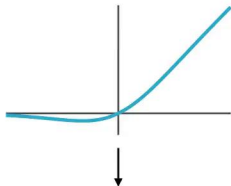
$$\text{relu}_\tau(x) := \int_0^x H_\tau(t) dt$$



Connection between ReLu and axiswise operators

Gating

$$\text{relu}_\tau(x) := H_\tau(x) \cdot x$$

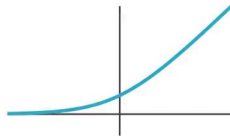


Similar structure to axiswise operators!

$$\text{sort}_\tau(\mathbf{x}) := \text{argsort}_\tau(\mathbf{x}) \mathbf{x} = P_\tau^*(\mathbf{x}) \mathbf{x}.$$

Integration

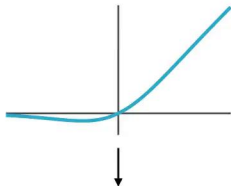
$$\text{relu}_\tau(x) := \int_0^x H_\tau(t) dt$$



Connection between ReLu and axiswise operators

Gating

$$\text{relu}_\tau(x) := H_\tau(x) \cdot x$$

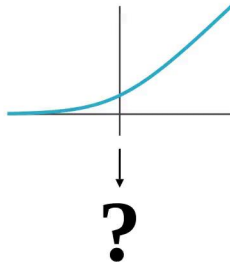


Similar structure to axiswise operators!

$$\text{sort}_\tau(\mathbf{x}) := \text{argsort}_\tau(\mathbf{x}) \mathbf{x} = P_\tau^*(\mathbf{x}) \mathbf{x}.$$

Integration

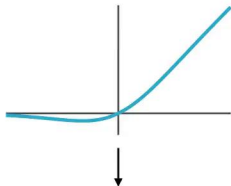
$$\text{relu}_\tau(x) := \int_0^x H_\tau(t) dt$$



Connection between ReLu and axiswise operators

Gating

$$\text{relu}_\tau(x) := H_\tau(x) \cdot x$$

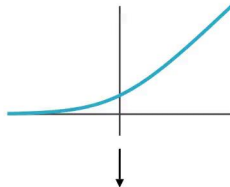


Similar structure to axiswise operators!

$$\text{sort}_\tau(\mathbf{x}) := \text{argsort}_\tau(\mathbf{x}) \mathbf{x} = P_\tau^*(\mathbf{x}) \mathbf{x}.$$

Integration

$$\text{relu}_\tau(x) := \int_0^x H_\tau(t) dt$$



Autodiff only needs the Jacobian!

```
P = stop_gradient(sj.argsort(x))
sorted_values = sj.take_along_axis(x, P)
```


Library overview

Axiswise	Elementwise	Logical	Selection	Modes
argmin	heaviside	logical_and	where	✓ smooth
min	round	logical_or	take_along_axis	✓ c0
argsort	sign	logical_xor	take	✓ c1
sort	abs	logical_not	choose	✓ c2
argquantile	relu	any	dynamic_index_in_dim	Methods
quantile	clip	all	dynamic_slice_in_dim	✓ Optimal Transport
argmedian	less		dynamic_slice	✓ SoftSort
median	less_equal			✓ NeuralSort
top_k	equal			✓ FastSoftSort
rank	not_equal			✓ SmoothSort
	isclose			✓ Sorting Network

Library overview

Appendix: Proof of smoothness of p-norm regularized projections.



Axiswise	Elementwise	Logical	Selection	Modes
argmin	heaviside	logical_and	where	✓ smooth
min	round	logical_or	take_along_axis	✓ c0
argsort	sign	logical_xor	take	✓ c1
sort	abs	logical_not	choose	✓ c2
argquantile	relu	any	dynamic_index_in_dim	Methods
quantile	clip	all	dynamic_slice_in_dim	✓ Optimal Transport
argmedian	less		dynamic_slice	✓ SoftSort
median	less_equal			✓ NeuralSort
top_k	equal			✓ FastSoftSort
rank	not_equal			✓ SmoothSort
	isclose			✓ Sorting Network

Library overview

Appendix: Proof of smoothness of p-norm regularized projections.

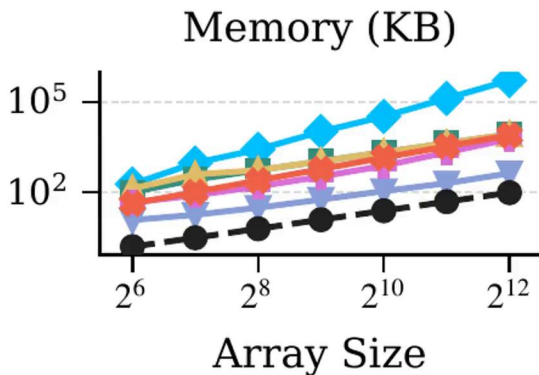
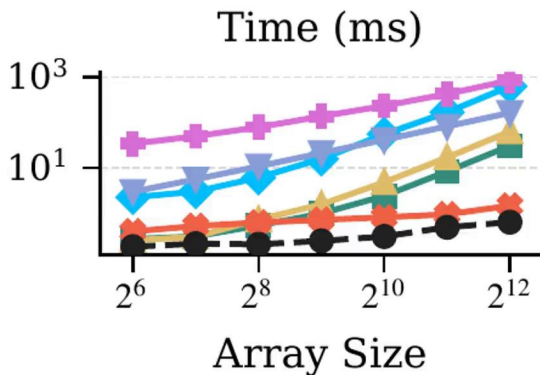


Axiswise	Elementwise	Logical	Selection	Modes
argmin	heaviside	logical_and	where	✓ smooth
min	round	logical_or	take_along_axis	✓ c0
argsort	sign	logical_xor	take	✓ c1
sort	abs	logical_not	choose	✓ c2
argquantile	relu	any	dynamic_index_in_dim	Methods
quantile	clip	all	dynamic_slice_in_dim	✓ Optimal Transport
argmedian	less		dynamic_slice	✓ SoftSort
median	less_equal			✓ NeuralSort
top_k	equal			✓ FastSoftSort
rank	not_equal			✓ SmoothSort
	isclose			✓ Sorting Network



Appendix: Elementwise operators are special case of axiswise operators.

Sorting

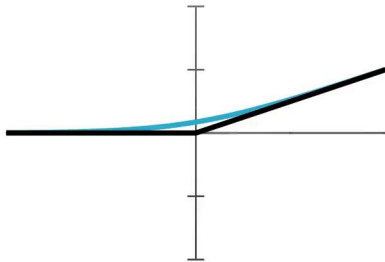


Reducing gradient biases via STE

Straight-through trick

$$f_{\text{STE}}(x) = \text{sg}(f(x)) + f_{\tau}(x) - \text{sg}(f_{\tau}(x))$$

— `sj.relu(x, mode="hard")`
— `sj.relu(x, mode="smooth")`

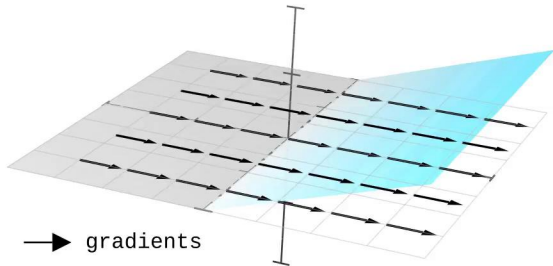


Reducing gradient biases via STE

Straight-through trick

$$f_{\text{STE}}(x) = \text{sg}(f(x)) + f_{\tau}(x) - \text{sg}(f_{\tau}(x))$$

`sj.st(sj.relu)(x)`

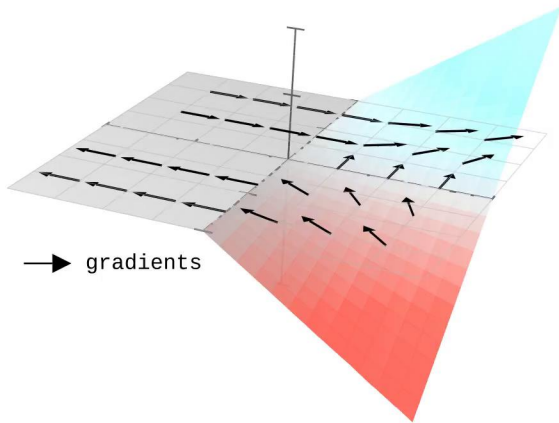


Reducing gradient biases via STE

Straight-through trick

$$f_{\text{STE}}(x) = \text{sg}(f(x)) + f_{\tau}(x) - \text{sg}(f_{\tau}(x))$$

$y * \text{sj.st}(\text{sj.relu}(x))$



Reducing gradient biases via STE

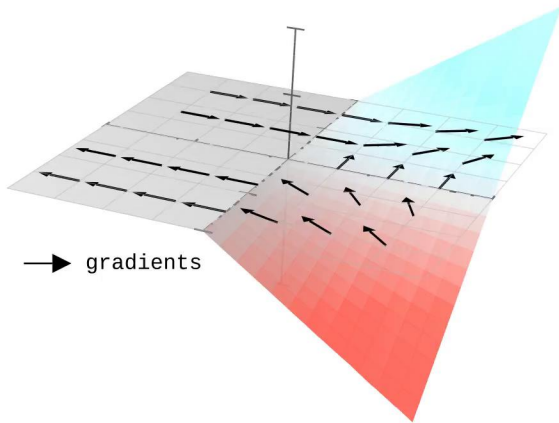
Straight-through trick

$$f_{\text{STE}}(x) = \text{sg}(f(x)) + f_{\tau}(x) - \text{sg}(f_{\tau}(x))$$

Product rule:

$$\nabla(f_{\text{STE}} \cdot g_{\text{STE}}) = \nabla f_{\tau} \cdot g + f \cdot \nabla g_{\tau}$$

$y * \text{sj.st}(\text{sj.relu}(x))$



Reducing gradient biases via STE

Straight-through trick

$$f_{\text{STE}}(x) = \text{sg}(f(x)) + f_{\tau}(x) - \text{sg}(f_{\tau}(x))$$

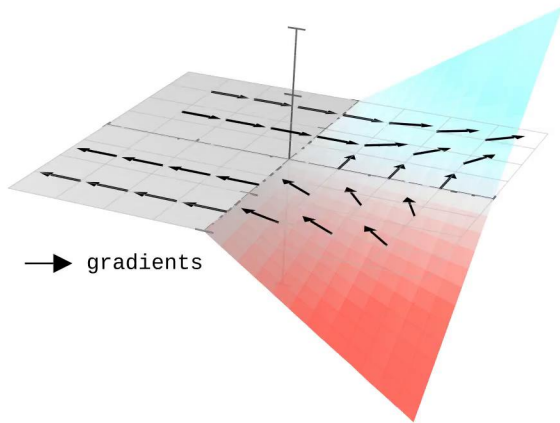
Product rule:

$$\nabla(f_{\text{STE}} \cdot g_{\text{STE}}) = \nabla f_{\tau} \cdot g + f \cdot \nabla g_{\tau}$$

Straight-through pitfall

$$(f \cdot g)_{\text{STE}} \neq f_{\text{STE}} \cdot g_{\text{STE}}$$

$$y * \text{sj.st}(\text{sj.relu}(x))$$



Reducing gradient biases via STE

Straight-through trick

$$f_{\text{STE}}(x) = \text{sg}(f(x)) + f_{\tau}(x) - \text{sg}(f_{\tau}(x))$$

Product rule:

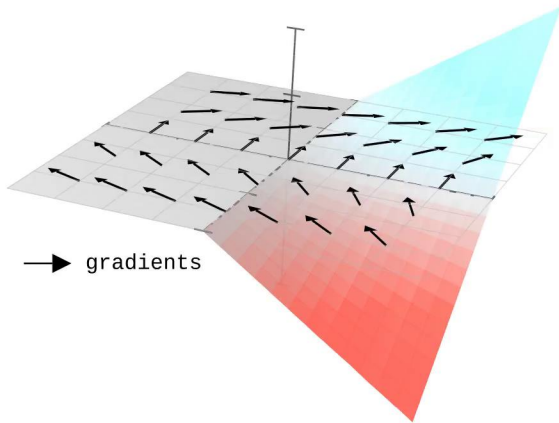
$$\nabla(f_{\text{STE}} \cdot g_{\text{STE}}) = \nabla f_{\tau} \cdot g + f \cdot \nabla g_{\tau}$$

Straight-through pitfall

$$(f \cdot g)_{\text{STE}} \neq f_{\text{STE}} \cdot g_{\text{STE}}$$

@sj.st

```
def f(x, y):  
    return y * sj.relu(x)
```



Thanks for all the fish.



Anselm Paulus



Georg Martius



Vít Musil



Sebastian Hoffmann